text: https://hackage.haskell.org/package/text

```
data Text
instance isString Text
instance isList Text
instance Eq Text
instance Ord Text
...

-- Basic
pack :: String -> Text
unpack :: Text -> String
empty :: Text
singleton :: Char -> Text
compareLength :: Text -> Int -> Ordering
cons :: Char -> Text -> Text
snoc :: Text -> Char -> Text
uncons :: Text -> Maybe (Char, Text)
unsnoc :: Text -> Maybe (Text, Char)
head :: Text -> Char
tail :: Text -> Text
init :: Text -> Text
last :: Text -> Char
null :: Text -> Bool
length :: Text -> Int
index :: Text -> Int -> Char

-- Utils
map :: (Char -> Char) -> Text -> Text
intercalate :: Text -> [Text] -> Text
intersperse :: Char -> Text -> Text
...
```

==========
======

Textshow

text-show: https://hackage.haskell.org/package/text-show

```
--
module TextShow

data Builder                                        --
instance Monoid, IsString

class TextShow a where
 showbPrec :: Int -> a -> Builder
 showb :: a -> Builder
 showbList :: [a] -> Builder

 showtPrec a -> Text
```

```
 showt :: a -> Text
 showtList :: [a] -> Text

 showtlPrec :: Int -> a -> Text                    --
 Data.Text.Lazy
 showtl :: a -> Text
 showtlList :: [a] -> Text

showbParen :: Bool -> Builder -> Builder
showtParen :: Bool -> Text -> Text
showtlParen :: Bool -> Text -> Text
showbCommaSpace :: Builder
showtCommaSpace :: Builder
showtlCommaSpace :: Builder
showbSpace :: Builder
showtSpace :: Text
showtlSpace :: Text


-- Builder
toText :: Builder -> Text
toLazyText :: Builder -> Text
toLazyTextWith :: Int -> Builder -> Text
toString :: Builder -> String
fromText :: Text -> Builder
fromLazyText :: Text -> Builder
fromString :: Text -> Builder
singleton :: Char -> Builder
flush :: Builder
...
```

==========

fmt

Formatting: https://hackage.haskell.org/package/formatting
fmt: https://hackage.haskell.org/package/fmt

main classes

```
-- From formatting: https://hackage.haskell.org/package/
formatting
newtype Builder

-- Value to Builder
class Buildable p where
    build :: p -> Builder

-- Output text
class FromBuilder a where
  fromBuilder :: Builder -> a

fmt :: FromBuilder b => Builder -> b
fmtLn :: FromBuilder b => Builder -> b
```

```
pretty :: (Buildable a, FromBuilder b) => a -> b
prettyLn :: (Buildable a, FromBuilder b) => a -> b

(+|) :: FromBuilder b => Builder -> Builder ->
b
(+||) :: FromBuilder b => Builder -> Builder -> b
(|+) :: (Buildable a, FromBuilder b) => a -> Builder ->
b
(||+) :: (Show a, FromBuilder b) => a -> Builder -> b

-- Basic
nameF :: Builder -> Builder -> Builder
indentF :: Int -> Builder -> Builder
indentF' :: Int -> Text -> Builder -> Builder
unwordsF :: (Foldable f, Buildable a) => f a -> Build
unlinesF :: (Foldable f, Buildable a) => f a -> Build

-- List
listF :: (Foldable f, Buildable a) => f a -> Builder
jsonListF :: (Foldable f, Buildable a) => f a -> Builder
blockListF :: (Foldable f, Buildable a) => f a ->
Builder
listF' :: (Foldable f, Buildable a) => (a -> Builder) ->
f a -> Builder
jsonListF' :: (Foldable f, Buildable a) => (a ->
Builder) -> f a -> Builder
blockListF'' :: (Foldable f, Buildable a) => Text -> (a
-> Builder) -> f a -> Builder
```

Instances

```
instance Buildable Int
instance Buildable Bool
instance Buildable Char
instance Buildable Text
...


instance FromBuilder Builder
instance FromBuilder String
instance FromBuilder Text
instance FromBuilder (IO ())
...
```

fmt
A module for **formatting** text
e.g. like printf
Uses **string builders**
i.e. string concatenators

e.g. Basic usage

```
{-# LANGUAGE OverloadedStrings #-}
```

```
import Data.Text(Text)
import Data.Text qualified as T
import Data.Text.IO qualified as TIO
import Fmt

main = do
  let
    name = "Yui" :: String
    value = 1 :: Int
    builder = "hello"+|name|+" value="+|value|+""

    " Execute builder
    str = fmt builder :: String
    txt = fmt builder :: Text
    builder2 = fmt builder :: Builder
  putStrLn str
  TIO.putStrLn txt
```

```
---- IMPORTANT: SHOULD have OverloadedStrings enabled
--
{-# LANGUAGE OverloadedStrings #-}
```

```
-- With OverloadedStrings
{-# LANGUAGE OverloadedStrings #-}
b = "a" :: Builder
fmt $ "value is"+|1|+""

-- Without OverloadedStrings
b = build "a"
fmt $ (build "value is")+|1|+(build "")
```

```
---- create a Builder
--
-- Builder is an ADT i.e. no explicit constructor
-- Builder is instantiated by the Buildable class
-- build: create Builder
build :: Buildable p => p -> Builder
build ("a"::String)
"a"::Builder              -- from literal with
OverloadedString
build 1                   -- from a Buildable type
```

```
Usually don't need to call build explicitly.
Called implicitly by concatenation
e.g. by |+

-- e.g.
b = build "a"
b = "a"|+""::Builder     -- same
```

```
b = build 1
b= 1|+""::Builder        -- same
```

```
-- Buildable instances
build ("a"::String)         -- explicitly from String
build ("a"::Text)           -- explicitly from
Data.Text.Text
build 1                     -- from Int
build 2.0
build True
data MyType = A deriving (Show)
build $ show MyType         --
...

-- e.g. Buildable for user defined type
data MyType = A
build MyType
MyType|+""
```

```
import Fmt.Interna.Core

instance Buildable MyType where
 build A = build "MyTypeA"
```

```
---- execute Builder
-- i.e. finalize and output text from Builder
--
-- Builder executed by FromBuilder class.
```

```
class FromBuilder a where
  fromBuilder :: Builder -> a

instance FromBuilder Builder
instance FromBuilder String
instance FromBuilder Text
instance FromBuilder (IO ())
...
```

```
-- fmt: execute Builder
fmt :: FromBuilder b => Builder -> b
fmtLn :: FromBuilder b => Builder -> b
pretty :: (Buildable a, FromBuilder b) => a -> b
prettyLn :: (Buildable a, FromBuilder b) => a -> b
fmt = Fmt.Internal.Core.FromBuilder        -- same

b = build "hello"
fmt $ b :: String               -- as String
fmt $ b :: Data.Text.Text       -- as Text
fmt $ b :: Builder              -- as Builder i.e.
```

unchanged


```
----  Concatenation
--
(+|) :: FromBuilder b => Builder -> Builder -> b

(+||) :: FromBuilder b => Builder -> Builder -> b
(|+) :: (Buildable a, FromBuilder b) => a -> Builder ->
b
(||+) :: (Show a, FromBuilder b) => a -> Builder -> b

"x="+|1|+""
"x="+|1|+"" :: Builder        -- as Builder
"x="+|1|+"" :: String         -- as String
"x="+|1|+"" :: Text

-- right associative
"x="+|1|+""
"x="+(|1|+"")    -- same
("x="+|1)|+""    -- ERROR. 1 not a Builder


-- Builder + Builder
"hello" +| "world"
"hello" +|| "world"      -- same
build 1 +| build 2       -- OK, but better to use |+

-- Buildable + Builder
1 |+ "=one"
build 1 +| "=one"        -- same
[1] |+ "=list of ones"   --

-- Show + builder
```

```
data MyType = A deriving (Show)
```

```
MyType ||+ "=MyType"                -- "A=MyType
build $ show MyType |+ "=MyType"    -- same
MyType |+ "=MyType"                 -- ERROR


"hello"+|"world\n"+|1|+"=one\n"+||MyType||+"=MyType\n"
```

```
helloworld
1=one
MyType=MyType
```

```
---- Formatter
--
-- basic
nameF :: Builder -> Builder -> Builder
indentF :: Int -> Builder -> Builder
indentF' :: Int -> Text -> Builder -> Builder
```

```
unwordsF :: (Foldable f, Buildable a) => f a -> Build
unlinesF :: (Foldable f, Buildable a) => f a -> Build
nameF "name" "Yui"        -- "name:Yui"
indentF 2 "a\nb\nc"
```

```
  a
  b
  c
```

```
indentF' 2 "name:" "a\nb\nc"
```

```
name:a
  b
  c
```

```
unwordsF ["a","b","c"]     -- "a b c"
unlinesF ["a","b","c"]     -- "a\nb\nc"

-- list
listF :: (Foldable f, Buildable a) => f a -> Builder
jsonListF :: (Foldable f, Buildable a) => f a -> Builder
blockListF :: (Foldable f, Buildable a) => f a ->
Builder
listF' :: (Foldable f, Buildable a) => (a -> Builder) ->
f a -> Builder
jsonListF' :: (Foldable f, Buildable a) => (a ->
Builder) -> f a -> Builder
blockListF'' :: (Foldable f, Buildable a) => Text -> (a
-> Builder) -> f a -> Builder
listF' ["a","b"]                    -- "[a,b]"
listF' (nameF "item") ["a","b"]     -- "[item: a\n, item
b\n]"
```

```
listF ["a","b"]
[a, b]

jsonListF ["a","b"]
[
  a
, b
]

blockListF ["a","b"]
- a
- b
```

```
... etc ...
```

Text.Blaze.Html: https://hackage.haskell.org/package/blaze-html

Usage: insert raw HTML
Use **preEscapedToHtml**

```
{-# LANGUAGE OverloadedStrings #-}
import Text.Blaze.HTML5 qualified as H

page :: H.Html
page = H.body $ do
  H.toHtml "<span>Hello</span>"                    --
BAD Shows literal "<span>"
  H.preEscapedToHtml "<span>Hello</span>"          --
GOOD
```